

Trabalho Prático 2, Parte 1

Pipeline de *Information Retrieval* e *Question Answering*
sobre artigos da Wikipedia

João Moura

Universidade do Minho, Mestrado em Engenharia Informática
Scripting no Processamento de Linguagem Natural (SPLN), 2025/26

Maio de 2026

Abstract

Este relatório descreve a implementação de uma *pipeline* de *Information Retrieval* (IR) e *Question Answering* (QA) construída sobre um corpus de artigos da Wikipedia. Como tema foram escolhidos os países do mundo (lista oficial das Nações Unidas), tendo sido construído um corpus de 196 artigos partidos em aproximadamente 9000 *chunks*. O sistema implementa um *retriever* híbrido que combina *BM25* (léxico) e *Sentence-BERT* (semântico) através de *Reciprocal Rank Fusion*, e dois sistemas de QA, um extractivo, com *fine-tuning* de DistilBERT sobre o dataset SQuAD, e outro abstrativo, baseado num esquema RAG (*Retrieval-Augmented Generation*) com o modelo *FLAN-T5*. Os resultados mostram que os dois paradigmas de QA são complementares: o extractivo destaca-se em respostas factoides com elevada precisão, enquanto o abstrativo é mais robusto em paráfrases, perguntas relacionais e em situações em que a informação não está disponível no corpus.

1 Introdução

O Trabalho Prático 2 da UC de SPLN tem como objetivo a implementação prática de uma *pipeline* de *Information Retrieval* e *Question Answering*, integrando os conceitos estudados ao longo do semestre: *word embeddings*, *transformers*, *fine-tuning* de modelos pré-treinados e técnicas de *retrieval* léxico e semântico.

Os requisitos do enunciado especificam três tarefas: (a) construção de um corpus com pelo menos 100 documentos sobre um tema à escolha; (b) implementação de um módulo *retriever* capaz de devolver os documentos mais relevantes para uma *query*; (c) implementação de duas estratégias de *Question Answering*, uma extractiva, com *fine-tuning* no SQuAD [4], e uma abstractiva, com *prompting* a um modelo generativo.

Optou-se por construir o corpus a partir de artigos da Wikipedia em inglês sobre os países do mundo. A escolha foi motivada por três fatores: (i) cumprimento confortável do requisito mínimo de documentos (193 Estados-membros da ONU mais alguns observadores); (ii) artigos de qualidade editorial elevada e estrutura relativamente uniforme; (iii) riqueza factual que permite formular perguntas variadas (geográficas, demográficas, históricas, políticas) para avaliação do sistema.

Todo o sistema está implementado num único *Jupyter notebook* (`tp2.ipynb`) com seis secções correspondentes às fases da *pipeline*: corpus, pré-processamento, *retriever*, QA extractivo, QA abstrativo e demonstração.

2 Construção do Corpus

O corpus foi construído a partir da biblioteca `wikipedia-api` (*wrapper* oficial da API REST da Wikipedia). Para cada país da lista de 196 países foi descarregado o artigo Wikipedia correspondente, com os seguintes cuidados:

[noitemsep] **User-Agent próprio**, em conformidade com a política da Wikipedia, identificando o projeto acadêmico. **Resolução de ambiguidades**: alguns títulos de países colidem com outras entidades (por exemplo, “Georgia” refere tanto o país do Cáucaso como o estado dos EUA). Foi criado um mapeamento manual de *overrides* para esses casos. **Filtragem de secções navegacionais**: secções como *See also*, *References*, *External links*, *Notes*, *Further reading* e *Bibliography* foram excluídas por não conterem conteúdo factual relevante. **Limpeza de texto**: remoção de marcadores de referência ([1], [2], etc.) e normalização de espaçamento. **Cache em disco**: cada artigo é guardado num ficheiro `.txt` próprio, permitindo retomar a recolha em caso de interrupção sem repetir pedidos à API.

Estatísticas do corpus bruto

Métrica	Valor
Países na lista	196
Artigos descarregados com sucesso	[PREENCHER]
Total de caracteres no corpus	[PREENCHER]
Total de palavras no corpus	[PREENCHER]
Média de caracteres por artigo	[PREENCHER]

Table 1: Estatísticas do corpus após recolha.

3 Pré-processamento e *Chunking*

Os artigos da Wikipedia têm tipicamente dezenas de milhares de caracteres, demasiado para passar diretamente a um modelo de QA, que num *transformer* típico está limitado a 512 *tokens* de contexto. Além disso, o *retriever* funciona melhor sobre passagens curtas e tematicamente coerentes do que sobre artigos completos.

A estratégia adotada foi a divisão por **parágrafos naturais** (separados por dupla quebra de linha no texto extraído da Wikipedia), com filtragem de parágrafos com menos de 30 palavras (geralmente cabeçalhos, frases soltas ou *stubs*). Cada *chunk* é guardado num único ficheiro `corpus.json` com a estrutura `{id, country, text}`.

Listing 1: Função de *chunking* por parágrafos.

```
def chunk_article(text, min_words=30):
    paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
    return [p for p in paragraphs if len(p.split()) >= min_words]
```

Esta abordagem foi preferida a outras alternativas (janela deslizante de tamanho fixo, divisão por frase) por duas razões: (i) preserva a coerência semântica do parágrafo original, que tipicamente trata um único tópico; (ii) tem implementação simples e reproduzível, sem hiperparâmetros sensíveis.

4 *Retriever* Híbrido

O módulo *retriever* foi implementado em três variantes, com o objetivo de comparar abordagens léxicas, semânticas e híbridas.

Métrica	Valor
Total de <i>chunks</i>	[PREENCHER]
Média de <i>chunks</i> por país	[PREENCHER]
Mediana de palavras por <i>chunk</i>	[PREENCHER]
Mínimo / máximo de palavras por <i>chunk</i>	[PREENCHER]

Table 2: Estatísticas após *chunking*.

4.1 BM25 (léxico)

A primeira abordagem usa **BM25** [7], uma função de pontuação que generaliza o TF-IDF e que continua a ser uma *baseline* forte em *retrieval*. A implementação usa a biblioteca `rank-bm25` (BM25Okapi). A tokenização é simples: *lowercase* e *split* por caracteres alfanuméricos (`re.findall(r"\w+", text.lower())`). Não foi feita remoção de *stopwords* porque o próprio BM25 ajusta o peso de termos frequentes através do IDF.

4.2 Sentence-BERT (semântico)

A segunda abordagem usa *embeddings* densos calculados com **Sentence-BERT** [3]. O modelo escolhido foi o `multi-qa-MiniLM-L6-cos-v1`, especificamente treinado para *retrieval* em contexto de QA, com 384 dimensões e tamanho reduzido (cerca de 80 MB), apropriado para correr em CPU.

Os *embeddings* de todos os *chunks* são pré-calculados uma única vez e guardados em `embeddings.npy`. Para uma *query*, calcula-se o *embedding* e ordenam-se os *chunks* por similaridade cosseno (que coincide com o produto interno, dado que os *embeddings* estão normalizados).

4.3 Híbrido - *Reciprocal Rank Fusion*

A terceira abordagem combina as *rankings* dos dois *retrievers* através de **Reciprocal Rank Fusion** (RRF) [8], definida por:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (1)$$

onde $R = \{\text{BM25}, \text{Sentence-BERT}\}$, $\text{rank}_r(d)$ é a posição do documento d na *ranking* do *retriever* r , e $k = 60$ é uma constante de suavização típica da literatura. A vantagem do RRF é combinar *rankings* em vez de *scores* brutos, evitando o problema das escalas diferentes (BM25 produz *scores* em $R_{\geq 0}$; *cosine* entre -1 e 1).

4.4 Análise comparativa

Para avaliar qualitativamente os três *retrievers*, foram usadas três *queries* representativas de padrões distintos de pergunta:

[noitemsep] **Q1.** *What is the capital of Portugal?* - factóide direto, com a *keyword* principal (“Portugal”) a aparecer literalmente no texto. **Q2.** *Which countries border Brazil?* - pergunta relacional, exige compreender o conceito de fronteira. **Q3.** *When did Germany reunify?* - envolve paráfrase, dado que o termo “reunify” não aparece no artigo (o texto usa “reunification” e “Berlin/Bonn Act of 1994”).

Os resultados observados ilustram bem as forças e fraquezas de cada abordagem:

Conclusões da análise:

Query	BM25 (top-1)	Semantic (top-1)	Híbrido (top-1)
Q1: Capital Portugal	<i>chunk</i> sobre Neanderthals (ocorrências literais de “Portugal” mas irrelevante)	<i>chunk</i> com “Lisbon as its capital”, correto	<i>chunk</i> com “Lisbon as its capital”, correto
Q2: Fronteiras Brasil	Uruguai (artigo sobre disputa territorial com o Brasil)	Brasil (<i>chunk</i> com lista de países vizinhos), correto	Uruguai, <i>ranking</i> arrastada pela concordância
Q3: Reunificação Alemanha	Camarões, Luxemburgo, Marshall Is. (mencionam “Germany” em contextos errados)	Três <i>chunks</i> sobre a Alemanha, incluindo o ato de reunificação, correto	<i>chunk</i> sobre capitulação Nazi (início da história da reunificação)

Table 3: Resultados dos três *retrievers* para as *queries* de teste.

[noitemsep]O **BM25** é o melhor quando a pergunta usa as mesmas palavras do texto, mas é facilmente enganado por frequência elevada de termos em contextos errados (caso Q1, em que o *chunk* sobre Neanderthals contém o termo “Portugal” mais vezes que o *chunk* pertinente). O **retriever semântico** é claramente superior em *queries* com paráfrases (Q3) e relacionais (Q2), pois consegue mapear o significado para além das palavras exatas. O **híbrido (RRF)** é robusto na maior parte dos casos, mas em Q2 produziu um resultado pior que o semântico isolado, porque o documento sobre Uruguai aparece bem posicionado em ambas as *rankings* e o RRF favorece o consenso entre os dois.

5 Question Answering Extractivo

A primeira estratégia de QA é *extractiva*: o modelo recebe uma pergunta e um contexto, e devolve uma *span* (sequência contígua de *tokens*) do contexto que constitui a resposta. Esta abordagem é típica em modelos baseados em BERT [1].

5.1 Modelo e *Fine-Tuning*

Foi escolhido o **distilbert-base-uncased** [2], uma versão destilada (40% mais pequena, 60% mais rápida) do BERT que preserva cerca de 97% do desempenho em tarefas *downstream*. O *fine-tuning* foi efetuado no SQuAD v1.1 [4], o dataset de referência para QA extractivo, composto por mais de 100 mil pares pergunta-resposta sobre artigos da Wikipedia.

Por restrições de tempo de computação (cerca de 30 min em GPU T4 do Google Colab), foi usado um subconjunto de 20 000 exemplos de treino e 500 de validação. O treino correu com a configuração detalhada na Tabela 4.

A tokenização usa a estratégia padrão para QA com SQuAD: o par (pergunta, contexto) é codificado conjuntamente, e em contextos longos é usado *stride* para criar múltiplas janelas sobrepostas, garantindo que a resposta correta cabe pelo menos numa delas. As posições de início e fim do *span* resposta nos *tokens* são calculadas a partir do *offset mapping*.

5.2 Resultados

A avaliação foi efetuada num subconjunto de 500 exemplos da partição *validation* do SQuAD, com as métricas oficiais Exact Match (EM) e F1.

A literatura reporta para DistilBERT com *fine-tuning* completo no SQuAD valores na ordem de EM ≈ 77 e F1 ≈ 85 . Com apenas 20 000 exemplos de treino (cerca de 23% do total) e 2

Hiperparâmetro	Valor
Modelo base	<code>distilbert-base-uncased</code>
Tamanho máximo de sequência	384
<i>Stride</i> (overflow)	128
<i>Learning rate</i>	3×10^{-5}
Épocas	2
<i>Batch size</i>	16
<i>Weight decay</i>	0.01
Precisão	FP16 (em GPU)

Table 4: Hiperparâmetros do *fine-tuning*.

Métrica	Valor
Exact Match (EM)	[PREENCHER]
F1 score	[PREENCHER]

Table 5: Resultados no SQuAD validation (500 exemplos).

épocas, é esperado um ligeiro decréscimo, mas com resultados perfeitamente apresentáveis para a tarefa.

5.3 Integração com o *retriever*

Para a inferência em *queries* reais, o sistema combina o *retriever* híbrido com o modelo extractivo: recupera os 3 *chunks* mais relevantes, executa o modelo QA sobre cada um, e escolhe a resposta com maior *score* de confiança. Esta estratégia permite recuperar quando o *retriever* não devolve o *chunk* ideal em primeiro lugar.

Listing 2: Inferência extractiva com retriever (top-3 chunks).

```
def extractive_qa(query, top_k=3):
    retrieved = hybrid_search(query, top_k=top_k)
    candidates = []
    for doc_id, retr_score in retrieved:
        chunk = corpus[doc_id]
        pred = qa_pipe(question=query, context=chunk["text"])
        candidates.append({
            "country": chunk["country"],
            "answer": pred["answer"],
            "qa_score": pred["score"],
        })
    return max(candidates, key=lambda c: c["qa_score"])
```

6 Question Answering Abstrativo (RAG)

A segunda estratégia segue o paradigma *Retrieval-Augmented Generation* (RAG) [5]: o *retriever* fornece o contexto a um modelo generativo, que produz uma resposta em linguagem natural, podendo parafrasear, sintetizar ou recusar responder.

6.1 Modelo

Foi usado o `google/flan-t5-base` [6], uma variante *instruction-tuned* do T5 com cerca de 250 M parâmetros. O FLAN-T5 foi treinado em mais de 1800 tarefas formuladas como instruções, o

que lhe confere capacidade *zero-shot* robusta para tarefas de QA quando recebe um *prompt* bem estruturado.

6.2 *Prompt* usado

A construção do *prompt* segue um padrão típico de RAG, com três cuidados específicos:

[noitemsep]Cada *chunk* no contexto é prefixado com [Nome do País] para o modelo perceber a proveniência da informação. Instrução explícita para responder apenas com base no contexto fornecido. Instrução explícita para dizer “*I don’t know*” quando o contexto não contém a resposta, comportamento desejável e que distingue esta estratégia da extractiva.

Listing 3: Estrutura do prompt RAG.

```
Answer the following question based only on the provided context.
If the context does not contain the answer, say "I don't know".

Context:
[Country 1] <chunk text>

[Country 2] <chunk text>

[Country 3] <chunk text>

Question: <user query>
Answer:
```

A geração foi configurada com *beam search* (4 *beams*) e limite de 80 *tokens* na resposta. O limite de *input* foi fixado em 1024 *tokens*.

7 Demonstração e Comparação dos Sistemas

Para avaliação qualitativa dos dois sistemas, foi construído um conjunto de 8 *queries* cobrindo padrões de pergunta distintos. A Tabela 6 apresenta os resultados.

Query	Extractivo	Abstrativo	Categoria
What is the capital of Portugal?	[PREENCHER]	[PREENCHER]	Factoide
What is the official language of Japan?	[PREENCHER]	[PREENCHER]	Factoide
What is the population of Iceland?	[PREENCHER]	[PREENCHER]	Numérico
Which countries border Brazil?	[PREENCHER]	[PREENCHER]	Relacional
When did India gain independence?	[PREENCHER]	[PREENCHER]	Histórico
When did Germany reunify?	[PREENCHER]	[PREENCHER]	Paráfrase
What currency is used in France?	[PREENCHER]	[PREENCHER]	Paráfrase
Who won the 2026 FIFA World Cup?	[PREENCHER]	[PREENCHER]	Armadilha

Table 6: Comparação extractivo vs. abstrativo nas *queries* de demonstração.

Observações esperadas:

[noitemsep]Em **factoides simples** (capital, língua), espera-se que ambos os sistemas acertem com facilidade, frequentemente com a mesma resposta. Em **perguntas numéricas** (população), o extractivo tende a devolver o número exato (*span* literal do texto); o abstrativo pode arredondar ou parafrasear. Em **perguntas relacionais** (fronteiras), o extractivo é limitado

a uma única *span* contígua e portanto perde a lista completa; o abstrativo consegue enumerar todos os países. Em **paráfrases**, ambos beneficiam do *retriever* semântico que devolve o *chunk* certo apesar de a *query* usar termos diferentes do texto. Na **pergunta-armadilha** (FIFA World Cup 2026, informação inexistente no corpus), o comportamento esperado é o ponto mais interessante: o modelo abstrativo idealmente responde “I don’t know” graças à instrução explícita no *prompt*, enquanto o extractivo, por construção, nunca devolve essa resposta e seleciona sempre alguma *span* do contexto, que neste caso será necessariamente errada. Esta diferença ilustra uma limitação estrutural dos modelos extractivos: ausência de mecanismo de abstenção.

8 Conclusões

Foi implementada uma *pipeline* completa de IR e QA sobre um corpus de aproximadamente 9000 *chunks* extraídos de artigos Wikipedia sobre os países do mundo. O *retriever* híbrido combina BM25 e Sentence-BERT, e os dois sistemas de QA cobrem as duas estratégias clássicas (extractiva via DistilBERT *fine-tuned*; abstractiva via RAG com FLAN-T5).

A análise comparativa permitiu três conclusões principais:

[noitemsep]O ***retriever* semântico** é claramente superior ao léxico em *queries* com paráfrases, mas o léxico mantém-se útil para *queries* com entidades nomeadas raras. O híbrido captura o melhor dos dois na *maioria* dos casos, embora em alguns possa ser superado pelo semântico isolado. Os dois sistemas de **QA** são complementares: o extractivo é mais preciso em factoides simples e oferece a vantagem de devolver a resposta na sua forma literal (útil para datas e nomes próprios), enquanto o abstrativo é mais flexível em perguntas relacionais e em paráfrases, e, mais importante, consegue abster-se quando não tem informação no contexto. A **combinação *retriever* + QA** é genuinamente sinérgica: o *retriever* maximiza *recall* (passa ao QA os candidatos prováveis), e o QA maximiza *precision* (escolhe a melhor resposta dentro desses candidatos). Esta separação de responsabilidades é o que torna a arquitetura robusta.

Limitações e Trabalho Futuro

[noitemsep]O modelo extractivo foi *fine-tuned* num subconjunto do SQuAD (20 000 exemplos) por restrições de tempo. Um treino completo (88 000 exemplos, 3 épocas) elevaria EM e F1 em alguns pontos. O modelo abstrativo FLAN-T5-*base* é relativamente pequeno (250 M parâmetros). Modelos maiores (FLAN-T5-*large* ou *xl*) ou modelos especializados em RAG dariam respostas de qualidade superior, em especial em perguntas multi-*hop*. A estratégia de *chunking* é simples (parágrafos). Estratégias mais sofisticadas, com *overlap* ou com pré-classificação por tipo de informação (geografia, demografia, história), poderiam melhorar a qualidade do *retrieval*. Não foi feita avaliação quantitativa do sistema completo (*end-to-end* em *queries* reais sobre o corpus de países) por inexistência de *ground truth* para esta tarefa específica. Uma extensão natural seria construir um pequeno conjunto de pares pergunta-resposta sobre os países e medir *accuracy* formalmente.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL*, 2019.
- [2] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108, 2019.

- [3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP*, 2019.
- [4] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *EMNLP*, 2016.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, 2020.
- [6] Hyung Won Chung, Le Hou, Shayne Longpre, et al. Scaling Instruction-Finetuned Language Models. arXiv:2210.11416, 2022.
- [7] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods. In *SIGIR*, 2009.